🚩 Challenge 1: 🔏 Decentralized Staking App

STAKING APP
SPEEDRUN ETHEREUM

👩‍💻 A superpower of Ethereum is allowing you, the builder, to create a simple set of rules that an adversarial group of players can use to work together. In this challenge, you create a decentralized application where users can coordinate a group funding effort. If the users cooperate, the money is collected in a second smart contract. If they defect, the worst that can happen is everyone gets their money back. The users only have to trust the code.

🏦 Build a `Staker.sol` contract that collects **ETH** from numerous addresses using a payable `stake()` function and keeps track of `balances`. After some `deadline` if it has at least some `threshold` of ETH, it sends it to an `ExampleExternalContract` and triggers the `complete()` action sending the full balance. If not enough **ETH** is collected, allow users to `withdraw()`.

🎛 Building the frontend to display the information and UI is just as important as writing the contract. The goal is to deploy the contract and the app to allow anyone to stake using your app. Use a `Stake(address, uint256)` event to list all stakes.

> "🧾 *Note: If you use named arguments in your event (e.g.* `event Stake(address indexed staker, uint256 amount)` *), you'll need to update* `/packages/nextjs/app/stakings/page.tsx` *to reference event parameters by their names instead of numeric indices."*

🌟 The final deliverable is deploying a Dapp that lets users send ether to a contract and stake if the conditions are met, then `yarn vercel` your app to a public webserver. Submit the url on SpeedRunEthereum.com!

> "💬 *Meet other builders working on this challenge and get help in the* *Challenge 1 Telegram*!"

Checkpoint 0: 📦 Environment 📚

Before you begin, you need to install the following tools:

- Node (v18 LTS)
- Yarn (v1 or v2+)
- Git

Then download the challenge to your computer and install dependencies by running:

```
npx create-eth@0.1.0 -e challenge-1-decentralized-staking challenge-1-decentralized-staking
cd challenge-1-decentralized-staking
```

> *"in the same terminal, start your local network (a blockchain emulator in your computer):"*

```
yarn chain
```

> *"in a second terminal window, 🛰 deploy your contract (locally):"*

```
cd challenge-1-decentralized-staking
yarn deploy
```

> *"in a third terminal window, start your 📱 frontend:"*

```
cd challenge-1-decentralized-staking
yarn start
```

📱 Open http://localhost:3000 to see the app.

> *"👩‍💻 Rerun* `yarn deploy` *whenever you want to deploy new contracts to the frontend. If you haven't made any contract changes, you can run* `yarn deploy --reset` *for a completely fresh deploy."*

🔏 Now you are ready to edit your smart contract `Staker.sol` in `packages/hardhat/contracts`

⚗️ At this point you will need to know basic Solidity syntax. If not, you can pick it up quickly by tinkering with concepts from 📑 Solidity By Example using 🏗 Scaffold-ETH-2. (In particular: global units, primitive data types, mappings, sending ether, and payable functions.)

Checkpoint 1: 🔏 Staking 💵

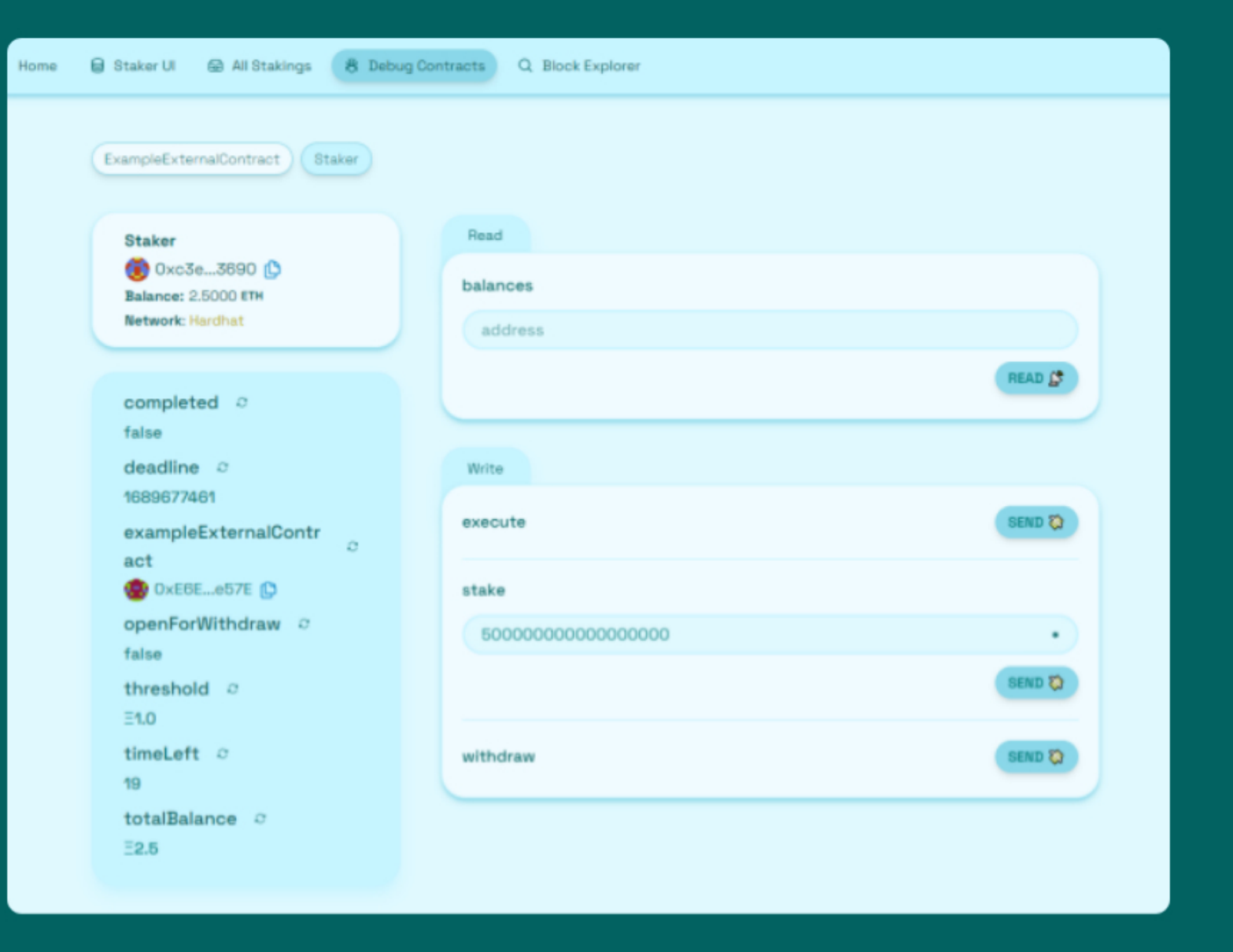
You'll need to track individual `balances` using a mapping:

```
mapping ( address => uint256 ) public balances;
```

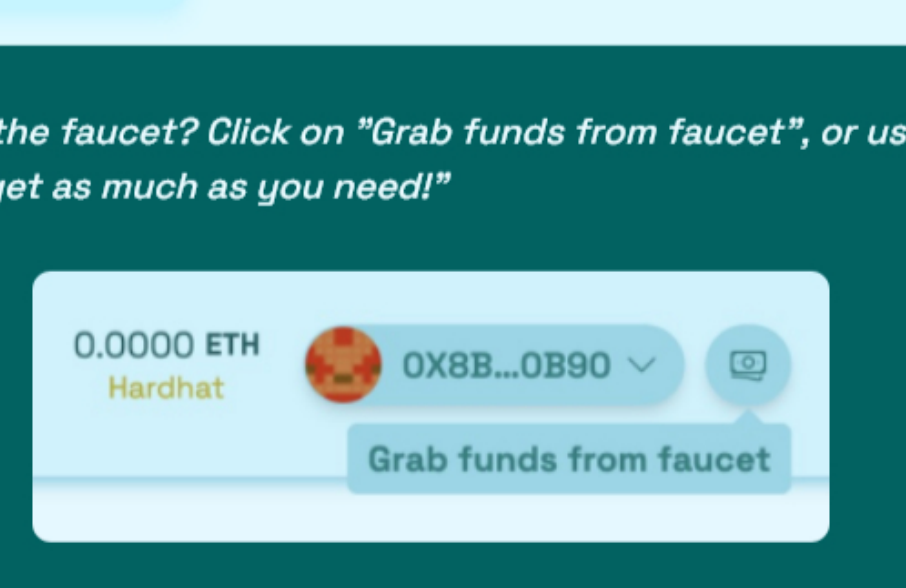
And also track a constant `threshold` at `1 ether`

```
uint256 public constant threshold = 1 ether;
```

> *"👩‍💻 Write your* `stake()` *function and test it with the* `Debug Contracts` *tab in the frontend."*



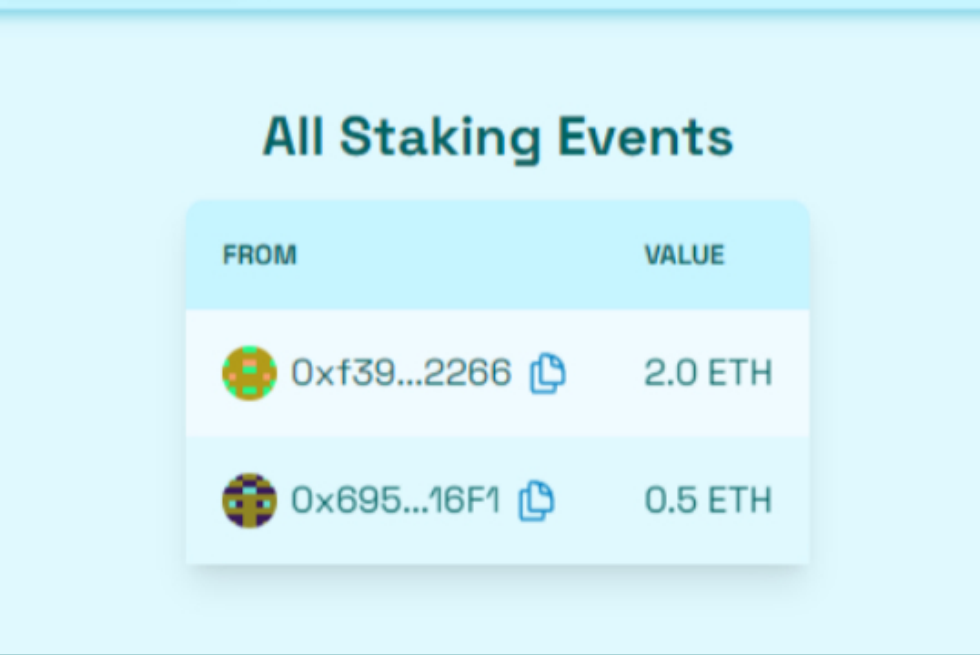
> *"💸 Need more funds from the faucet? Click on "Grab funds from faucet", or use the Faucet feature at the bottom left of the page to get as much as you need!"*



> *"✏ Need to troubleshoot your code? If you import* `hardhat/console.sol` *to your contract, you can call* `console.log()` *right in your Solidity code. The output will appear in your* `yarn chain` *terminal."*

🥅 Goals

- ⬛ Do you see the balance of the `Staker` contract go up when you `stake()`?
- ⬛ Is your `balance` correctly tracked?
- ⬛ Do you see the events in the `Stake Events` tab?



All Staking Events

FROM	VALUE
0xf39...2266	2.0 ETH
0x695...16F1	0.5 ETH

Checkpoint 2: 🔬 State Machine / Timing ⏱

State Machine

> *"⚙️ Think of your smart contract like a state machine. First, there is a **stake** period. Then, if you have gathered the* `threshold` *worth of ETH, there is a **success** state. Or, we go into a **withdraw** state to let users withdraw their funds."*

Set a `deadline` of `block.timestamp + 30 seconds`

```
uint256 public deadline = block.timestamp + 30 seconds;
```

👨‍🏫 Smart contracts can't execute automatically, you always need to have a transaction execute to change state. Because of this, you will need to have an `execute()` function that *anyone* can call, just once, after the `deadline` has expired.

> *"👩‍💻 Write your* `execute()` *function and test it with the* `Debug Contracts` *tab"*

> *"Check the* `ExampleExternalContract.sol` *for the bool you can use to test if it has been completed or not. But do not edit the* `ExampleExternalContract.sol` *as it can slow the auto grading."*

If the `address(this).balance` of the contract is over the `threshold` by the `deadline`, you will want to call: `exampleExternalContract.complete{value: address(this).balance}()`

If the balance is less than the `threshold`, you want to set a `openForWithdraw` bool to `true` which will allow users to `withdraw()` their funds.

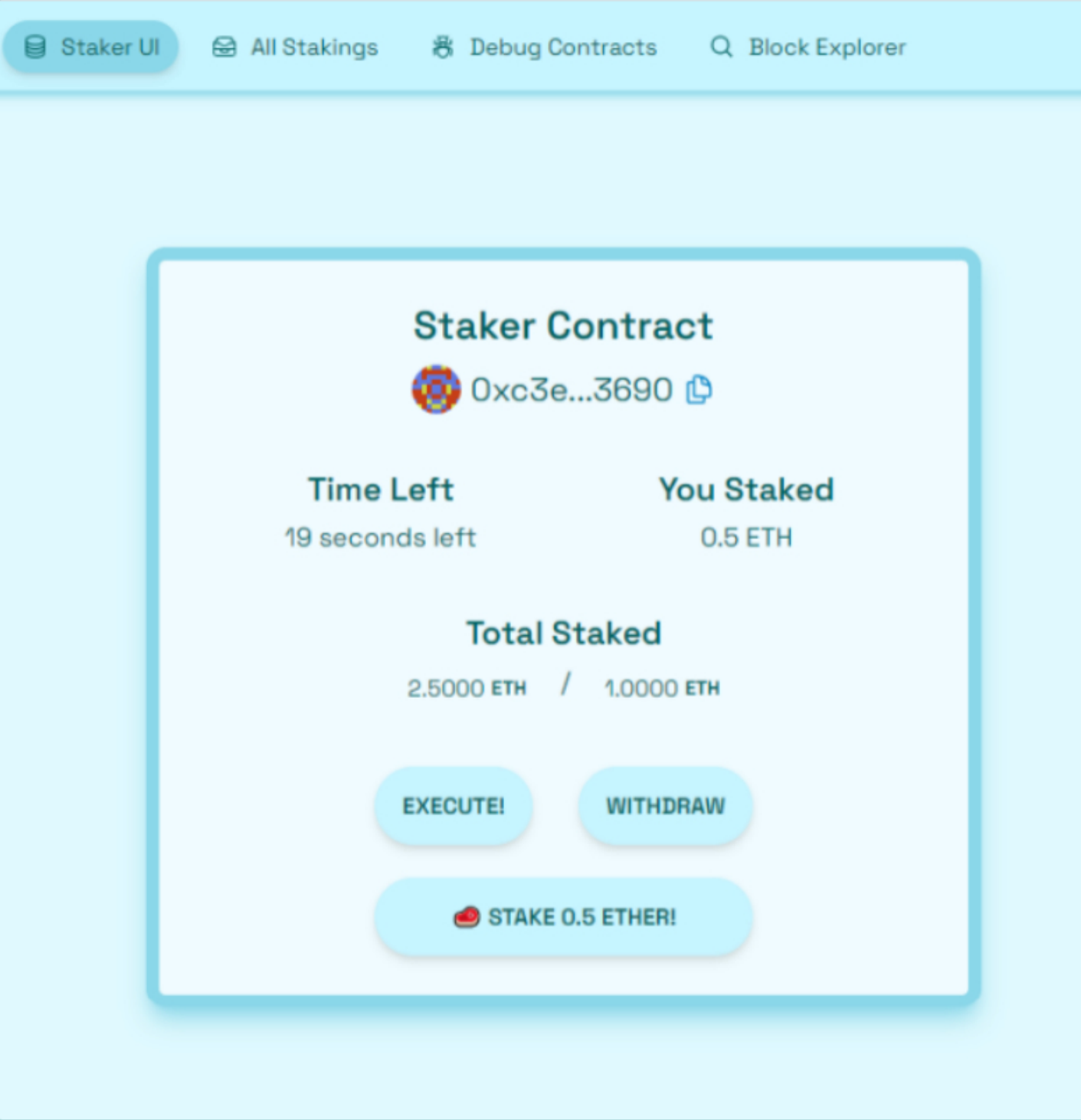
Timing

You'll have 30 seconds after deploying until the deadline is reached, you can adjust this in the contract.

> *"👩‍💻 Create a* `timeLeft()` *function including* `public view returns (uint256)` *that returns how much time is left."*

⚠️ Be careful! If `block.timestamp >= deadline` you want to `return 0;`

⏳ *"Time Left"* will only update if a transaction occurs. You can see the time update by getting funds from the faucet button in navbar just to trigger a new block.



> *"👩‍💻 You can call* `yarn deploy --reset` *any time you want a fresh contract, it will get re-deployed even if there are no changes on it. You may need it when you want to reload the "Time Left" of your tests."*

Your `Staker UI` tab should be almost done and working at this point.

🥅 Goals

- ⬛ Can you see `timeLeft` counting down in the `Staker UI` tab when you trigger a transaction with the faucet button?
- ⬛ If enough ETH is staked by the deadline, does your `execute()` function correctly call `complete()` and stake the ETH?
- ⬛ If the threshold isn't met by the deadline, are you able to `withdraw()` your funds?

Checkpoint 3: 💵 Receive Function / UX 🙎

🎀 To improve the user experience, set your contract up so it accepts ETH sent to it and calls `stake()`. You will use what is called the `receive()` function.

> *"Use the receive() function in solidity to "catch" ETH sent to the contract and call* `stake()` *to update* `balances`*."*

🥅 Goals

- ⬛ If you send ETH directly to the contract address does it update your `balance` and the `balance` of the contract?

⚔️ Side Quests

- ⬛ Can `execute()` get called more than once, and is that okay?
- ⬛ Can you stake and withdraw freely after the `deadline`, and is that okay?
- ⬛ What are other implications of *anyone* being able to withdraw for someone?

🐸 It's a trap!

- ⬛ Make sure funds can't get trapped in the contract! **Try sending funds after you have executed! What happens?**
- ⬛ Try to create a modifier called `notCompleted`. It will check that `ExampleExternalContract` is not completed yet. Use it to protect your `execute` and `withdraw` functions.

⚠️ Test it!

- Now is a good time to run `yarn test` to run the automated testing function. It will test that you hit the core checkpoints. You are looking for all green checkmarks and passing tests!

Checkpoint 4: 💾 Deploy your contract! 🛰

📡 Edit the `defaultNetwork` to your choice of public EVM networks in `packages/hardhat/hardhat.config.ts`

🔐 You will need to generate a **deployer address** using `yarn generate` This creates a mnemonic and saves it locally.

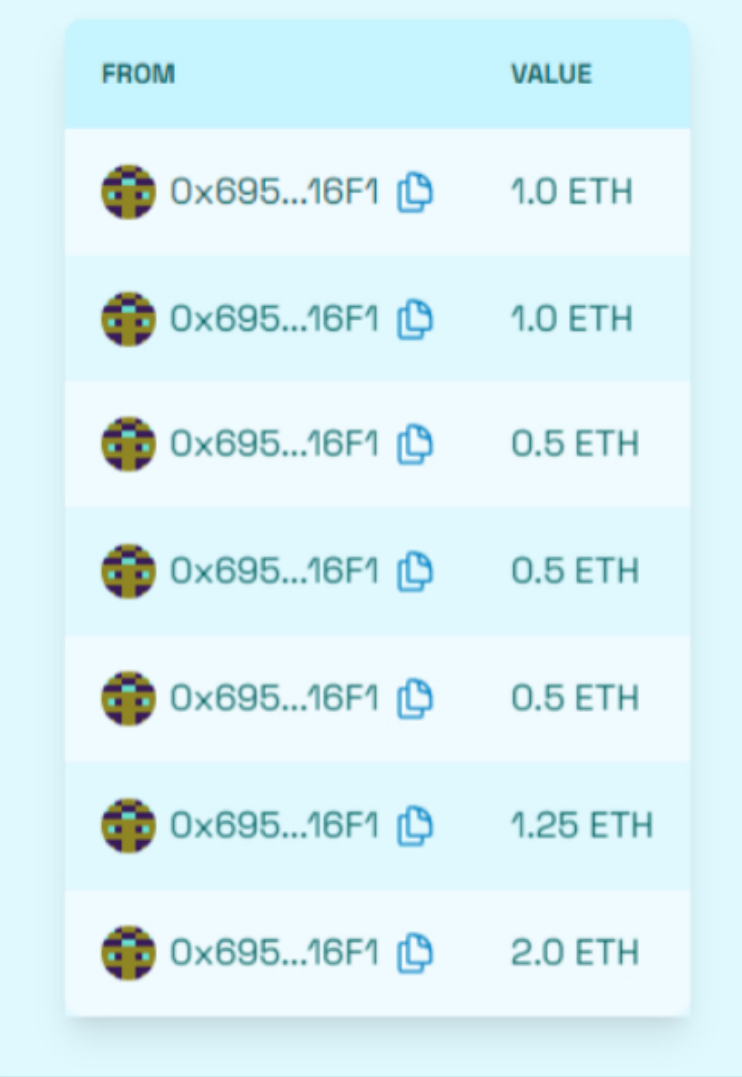
👩‍🚀 Use `yarn account` to view your deployer account balances.

⛽️ You will need to send ETH to your deployer address with your wallet, or get it from a public faucet of your chosen network.

> *"📝 If you plan on submitting this challenge, be sure to set your* `deadline` *to at least* `block.timestamp + 72 hours`*"*

🚀 Run `yarn deploy` to deploy your smart contract to a public network (selected in `hardhat.config.ts`)

> *"💬 Hint: You can set the* `defaultNetwork` *in* `hardhat.config.ts` *to* `sepolia` *or* `optimismSepolia` *OR you can* `yarn deploy --network sepolia` *or* `yarn deploy --network optimismSepolia`*."*



All Staking Events

FROM	VALUE
0x695...16F1	1.0 ETH
0x695...16F1	1.0 ETH
0x695...16F1	0.5 ETH
0x695...16F1	0.5 ETH
0x695...16F1	0.5 ETH
0x695...16F1	1.25 ETH
0x695...16F1	2.0 ETH

> *"💬 Hint: For faster loading of your "Stake Events" page, consider updating the* `fromBlock` *passed to* `useScaffoldEventHistory` *in* `packages/nextjs/app/stakings/page.tsx` *to* `blocknumber - 10` *at which your contract was deployed. Example:* `fromBlock: 3750241n` *(where* `n` *represents its a BigInt). To find this blocknumber, search your contract's address on Etherscan, find the* `More Info` *box, find the* `Contract Creator` *line, click on the* `txn`*, and find the value for* `Block`*."*

Checkpoint 5: 🚢 Ship your frontend! 🚁

✏️ Edit your frontend config in `packages/nextjs/scaffold.config.ts` to change the `targetNetwork` to `chains.sepolia` (or `chains.optimismSepolia` if you deployed to OP Sepolia)

💻 View your frontend at http://localhost:3000/staker-ui and verify you see the correct network.

📡 When you are ready to ship the frontend app...

📦 Run `yarn vercel` to package up your frontend and deploy.

> *"You might need to log in to Vercel first by running* `yarn vercel:login`*. Once you log in (email, GitHub, etc), the default options should work."*

> *"If you want to redeploy to the same production URL you can run* `yarn vercel --prod`*. If you omit the* `--prod` *flag it will deploy it to a preview/test URL."*

> *"Follow the steps to deploy to Vercel. It'll give you a public URL."*

> *"🦊 Since we have deployed to a public testnet, you will now need to connect using a wallet you own or use a burner wallet. By default 🔥* `burner wallets` *are only available on* `hardhat`*. You can enable them on every chain by setting* `onlyLocalBurnerWallet: false` *in your frontend config (* `scaffold.config.ts` *in* `packages/nextjs/` *)"*

Configuration of Third-Party Services for Production-Grade Apps.

By default, 🏗 Scaffold-ETH 2 provides predefined API keys for popular services such as Alchemy and Etherscan. This allows you to begin developing and testing your applications more easily, avoiding the need to register for these services. This is great to complete your **SpeedRunEthereum**.

For production-grade applications, it's recommended to obtain your own API keys (to prevent rate limiting issues). You can configure these at:

- 🔷 `ALCHEMY_API_KEY` variable in `packages/hardhat/.env` and `packages/nextjs/.env.local`. You can create API keys from the Alchemy dashboard.
- 📃 `ETHERSCAN_API_KEY` variable in `packages/hardhat/.env` with your generated API key. You can get your key here.

> *"💬 Hint: It's recommended to store env's for nextjs in Vercel/system env config for live apps and use .env.local for local testing."*

Checkpoint 6: 📜 Contract Verification

Run the `yarn verify --network your_network` command to verify your contracts on etherscan 🛰

👉 Search this address on Sepolia Etherscan (or Optimism Sepolia Etherscan if you deployed to OP Sepolia) to get the URL you submit to 🏃‍♀️ SpeedRunEthereum.com.

> *"🏃 Head to your next challenge here."*

> *"💬 Problems, questions, comments on the stack? Post them to the 🏗 scaffold-eth developers chat"*

Note: The staking page is in `/packages/nextjs/stakings/page.tsx.tsx`

View on GitHub

Related guides

Beyond APY: Building Sustainable Tokenomics for Your Staking Protocol	Time-Weighted Staking Rewards: Aligning Incentives in DeFi Protocols
Scalable Solidity Staking: O(1) Reward Distribution	A Developer's Guide to Liquid Staking Tokens (LSTs)